Architecturally Significant Requirements (ASRs)

1. Purpose

This document defines the Architecturally Significant Requirements (ASRs) and Non-Functional Requirements (NFRs) for the iDaVIE kernel and plug-in system, owned by Sub-team Apaties I. Requirements are derived from ISO/IEC 25010:2023 maintainability sub-characteristics and translated into concrete, testable statements that drive architecture decisions and CI enforcement.

2. Architecturally Significant Requirements (ASRs)

Each ASR maps directly to one ISO/IEC 25010:2023 maintainability sub-characteristic. All ASRs are enforceable — each has a defined verification method and is tracked in the CI pipeline from Day 2.

2.1 Modularity

Modularity	Degree to which a change to one component has minimal impact on others

Currently lack of modularity is a serious issue within the codebase likely created due to years of mounting technical debt. Architecture is tightly coupled with many god-classes. This makes changes risky and expensive: a modification to one component frequently forces changes (and recompilation) across many others, increases the chance of introducing defects, and makes the system difficult to analyse, modify and test. The requirements below directly target this problem by enforcing loose coupling, interface-based boundaries, and the elimination of dependency cycles.

Requirement	Verified by
No circular dependencies shall exist between any two top-level components Any circular dependencies currently in the codebase should be replaced with dependency inversion.	NDepend CQLinq rule on every PR — fail = merge blocked
A change to one plug-in (e.g. FitsReader) shall require zero recompilation of any other plug-in	Manual verification + CI build isolation test
CBO must be ≤14 for domain classes and ≤25 for orchestrators	Understand CK metrics snapshot at Day 2 and Day 13
Dependency cycles at namespace and assembly level must be 0	NDepend cycle detection - fail = merge blocked
All cross-component calls shall be made through an interface No component shall reference the concrete implementation type of another top-level component.	NDepend CQLinq rule asserting that methods on a public component boundary are only invoked via interface types — fail = merge blocked

2.2 Analysability

Analysability	Effectiveness of assessing change impact and identifying parts to modify

Requirement	Verified by
Cyclomatic complexity per method must not exceed 10 as outlined by IEEE.	SonarQube complexity gate on every PR
Classes exceeding 200 LOC, and Methods exceeding 30 LOC are marked for manual review Requiring manual review of lengthy classes and methods aims to ensure they remain within scope and don't lower readability.	SonarQube LOC rule — fail = merge blocked
Every public API boundary must have documented contract This creates a layer of abstraction and independence between the iDaVIE code base and the users of its API.	Manual Verification
Code duplication must not exceed 5% across domain classes - As listed by SWEBOK: To avoid the problem of code clones, developers should encapsulate reusable code fragments into well-structured libraries or components.	SonarQube duplication report

2.3 Modifiability

Modifiability	Degree to which a product can be modified without introducing defects

Requirement	Verified by
Replacing Unity version requires changes only in the Infrastructure layer — zero Domain or Application changes This enables easier migration to Unity 6 and future Unity versions. Separating Domain and Application logic also makes migrating to other game engines easier if required.	Architecture fitness function in CI (ArchUnit-style)
A new C/C++ plug-in can be added without modifying existing kernel code (Open-Closed Principle) Allo	Design review + Test suite
Maintain CodeScene Health Score of 8-10 (Healthy) Ensures low friction code changes. Code is consistently easy to read and modify.	CodeScene dashboard
Propagation cost (DV8) must decrease by ≥30%	DV8 sprint boundary snapshot

2.4 Testability

Testability	Effectiveness of establishing test criteria and performing tests

Requirement	Verified by
Every domain class must have zero direct UnityEngine imports Creating this separation allows domain code to be tested without requiring a full Unity build. This removes a lot of testing overhead by not requiring the Unity Editor or PlayMode runtime to spin up for every test. Tests can run as plain .NET unit tests against the domain logic directly, so they execute in milliseconds instead of waiting on a full engine load. This also makes tests more deterministic and makes producing test harnesses easier.	NDepend layer rule — fail = merge blocked
Branch coverage ≥70% on all domain and application layer classes	SonarQube coverage gate
Every public interface must be mockable with no Unity runtime present	Edit-mode unit test suite in CI

Interface size must not exceed 7 public members (ISP compliance)	NDepend interface size rule

3. Kernel Non-Functional Requirements (NFRs)

The following NFRs apply specifically to the kernel boundary and plug-in host layer. They govern runtime behaviour, operational resilience, and ABI contract stability.

NFR	Requirement	Measured by
Load time	Kernel initialises in under 3 seconds on target hardware	BenchmarkManager timing log
Plug-in isolation	A crash in one plug-in must not bring down the kernel or other plug-ins	Contract test suite — fault injection
Hot-reload	A plug-in can be unloaded and reloaded without restarting the application	Manual test + CI scenario script
Observability	Every plug-in call is logged with timing and error codes via structured logging	Log output verification in CI
ABI stability	No breaking ABI change within a major version — semantic versioning enforced	NDepend ABI versioning rules
Platform portability	No direct kernel32 calls — NativePluginLoader must be cross-platform	CI build matrix (Windows + Linux)

4. Stakeholder Expectations

Medium-term

- Switch rendering between emission and absorption
- Visualise particle / sparse multiparameter datasets (e.g. simulations)
- Scripting API for smooth camera fly-through videos
- Movable projection plane showing a 2D cross-section in a separate window
- Separate the visualisation tool from the analysis tools (plugin-based)

Long-term

- Integrate a Python console into the application
    - Spec created by Team 6
- Add visualisation modes such as isocontours
- Multiplayer / spectator mode (controller drives, others follow)
- Integrate a VO system for retrieving images from catalogues in other wavelengths

- Visualise multiple cubes at once, or a time-series
- Save application state / workspace (as in CARTA)
    - Fully implemented by Team 7

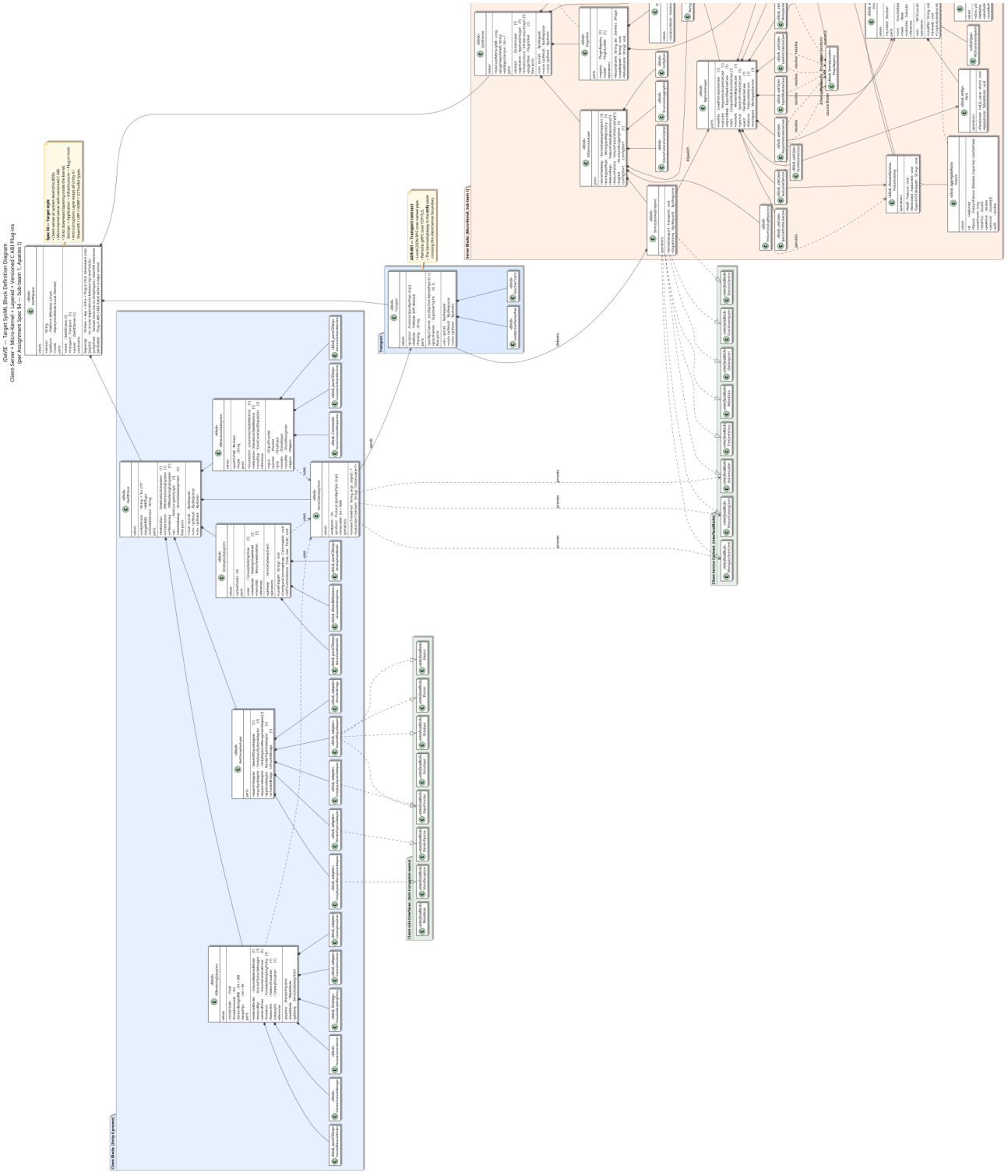
5. Requirements Traceability

All ASRs, NFRs and Stakeholder expectations listed above trace directly to the following sources:

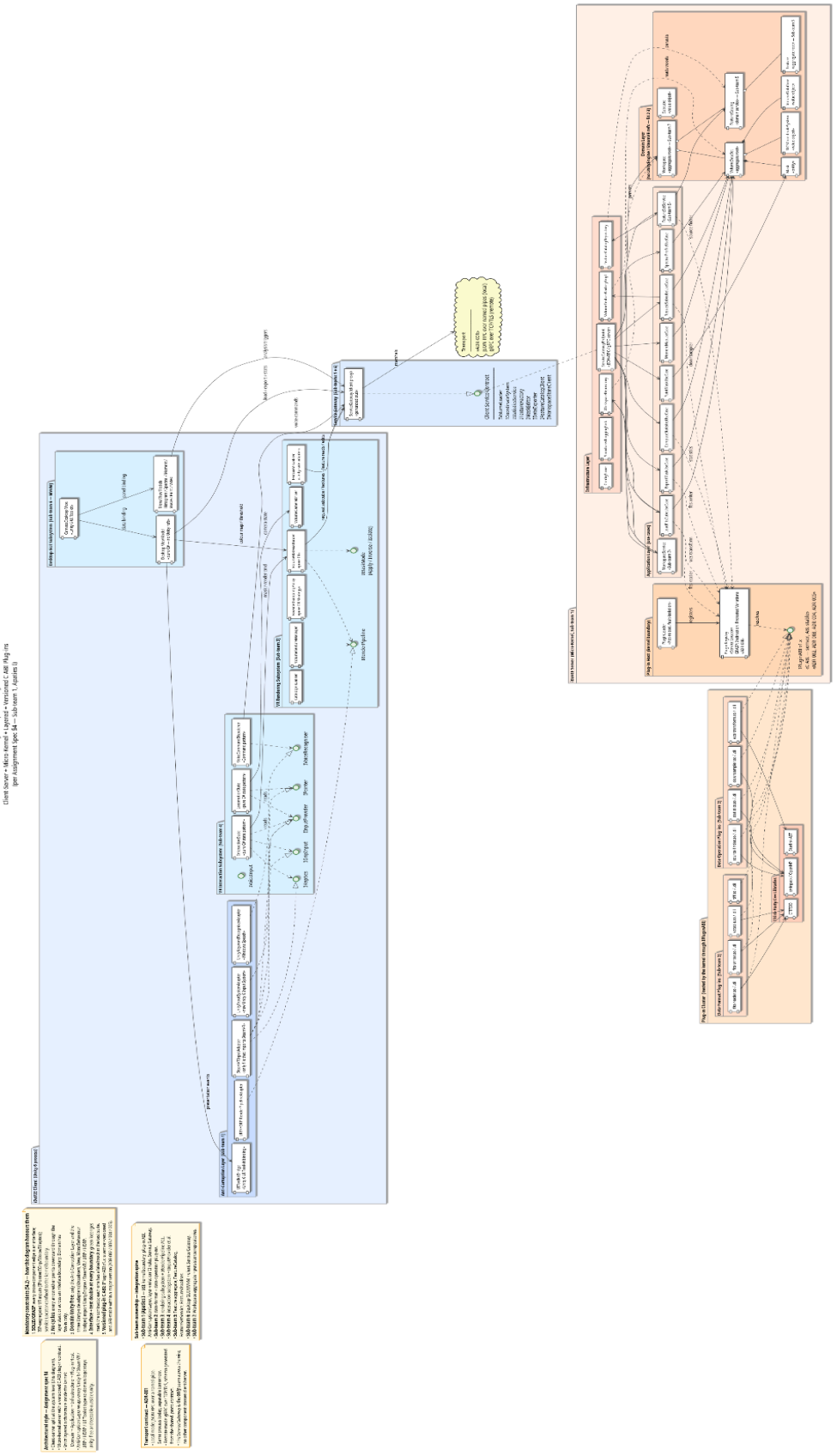
- ISO/IEC 25010:2023 — Modularity, Analysability, Modifiability, Testability sub-characteristics
- iDaVIE Assessment Specification — Section 4.2 Mandatory Architectural Constraints
- iDaVIE Assessment Specification — Section 6.1 Architecture and Micro-kernel Core
- iDaVIE codebase Day 2 baseline — identified god-classes, singletons, and Unity lifecycle pollution
- iDaVIE Future plans docs — Stakeholder Expectations
- SWEBOK Guide V4.0

Top-level component diagram (UML)

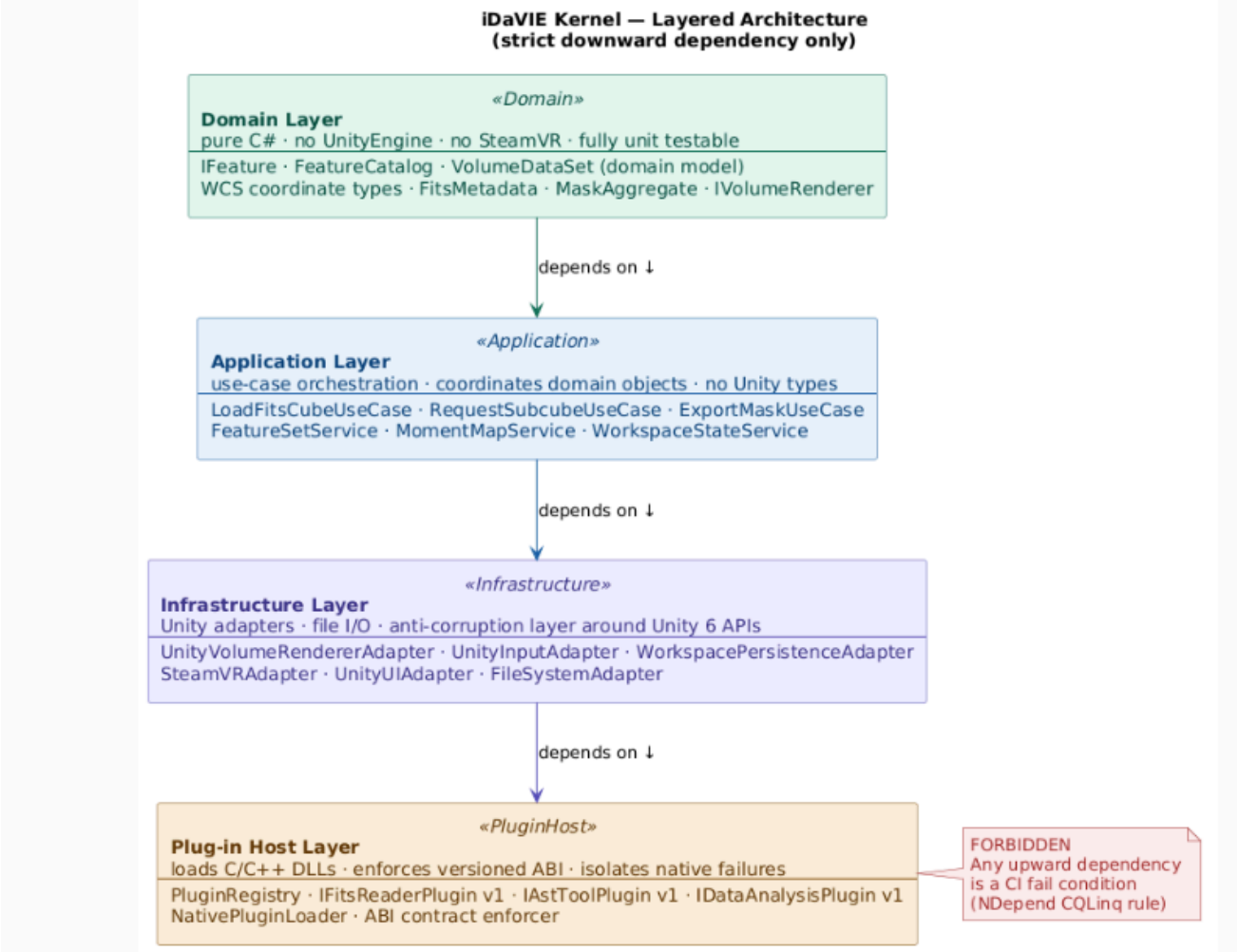
Client Server BDD



Client Server Component



The layered architecture inside the kernel



Domain is the core; pure business logic, no Unity, no plugins. Everything else knows about it, but it knows about nothing.

Application sits just below and orchestrates use cases - "load this FITS file", "export this mask". It calls Domain objects to do the actual work.

Infrastructure is where Unity lives. It wraps Unity and SteamVR APIs in adapters so the layers above never directly touch Unity code. This is what makes the Unity 5 to Unity 6 migration manageable, you only change the adapters.

Plug-in Host is at the bottom and handles the C/C++ DLLs. It loads FitsReader, AstTool and DataAnalysis through a versioned contract so they can be swapped or updated independently.

The golden rule: **lower layers never import upper layers**. Domain never knows Infrastructure exists. If that rule breaks, the CI pipeline blocks the merge automatically.

C ABI for plug-ins

iDaVIE Kernel & Plug-in System

Application Binary Interface (ABI) Specification

Owner	Sub-team 1 — Architecture and Micro-kernel Core	**Status**	**DRAFT v0.1.0**
Date	22 May 2026	**ABI Stable from**	v1.0.0 (Sprint 1 review, 22 May 2026)

1 Purpose

This document defines the Application Binary Interface (ABI) contract for all native C/C++ plug-ins that integrate with the iDaVIE micro-kernel. It establishes fixed-width type mappings across the native/managed boundary, a structured error model, threading guarantees, memory ownership rules, and a compatibility versioning policy. All requirements trace to ISO/IEC 25010:2023 (Modularity, Modifiability) and to iDaVIE Assessment Specification Section 4.2 Constraint 5.

2 Type Widths

All types that cross the ABI boundary must have an identical in-memory width on both sides. The table below documents every type that requires explicit handling. Using the fixed-width alternatives in the Fix column is mandatory for all exported signatures.

Native (C/C++) type	C# type (default marshalling)	Problem	Fix
`bool (1 byte, impl-defined)`	`Bool (4 bytes — Win32 BOOL)`	Marshaller writes/reads 4 bytes; native writes 1. Upper 3 bytes are stack noise.	Use int32_t on both sides, or keep bool on C# and add [MarshalAs(UnmanagedType.I1)].
`long (4 bytes Windows, 8 bytes Linux/macOS)`	`long (always 8 bytes)`	Width drift across platforms. Corrupts opposite platforms depending on which side allocates.	Use int64_t native and long C# — identical 8-byte layout everywhere. Ban long from any exported signature.
`int (≥16 bits, typically 4)`	`int (always 4 bytes)`	Usually agrees, but not guaranteed by the C standard.	Use int32_t native and int C#. Fixed-width discipline.
`size_t / ptrdiff_t (pointer-width)`	`IntPtr / UIntPtr`	Width depends on 32 vs 64-bit build; awkward to bind.	Use int64_t native and long C#. Use IntPtr only for opaque handles.
`char (1 byte, signedness impl-defined)`	`char (2 bytes — UTF-16)`	Totally different concepts. Naive marshalling silently mis-encodes strings.	For raw bytes: int8_t/uint8_t ↔ sbyte/byte. For text: const char* (UTF-8) ↔ [MarshalAs(UnmanagedType.LPUTF8Str)] string.

Native (C/C++) type	C# type (default marshalling)	Problem	Fix
`wchar_t(2 bytes Windows,4 bytes Linux/macOS)`	`char[] via [MarshalAs(LPWStr)](2 bytes)`	Width drift; Linux/macOS layouts disagree.	Do not use wchar_t at the boundary. Pin to UTF-8 const char*.
`enum(impl-defined underlying type)`	`enum(defaults to int)`	Underlying width can drift; reordering values is a silent ABI break.	Use enum class : int32_t { … } in C++, and enum E : int in C#. Freeze numeric values within a major version.
`struct with implicit padding`	`[StructLayout(LayoutKind.Sequential, Pack=N)]`	Padding can be computed differently on each side. (Example: SourceStats.spectralProfileSize 4-vs-8 bug.)	Add explicit _padding[N] fields, static_assert(sizeof(...) == N) in C++, and Debug.Assert(Marshal.SizeOf<T>() == N) in C#.
`void* / T*(raw pointer)`	`IntPtr`	Same width on a given build, but type-erased — all handles look the same.	Use only for opaque handles (typedef struct idavie_fits idavie_fits_t;). Never expose raw T* data buffers in the public API surface.
`bool[] / array of bool`	`bool[]`	Each side computes a different array stride.	Use int32_t[] ↔ int[].

3 Error Model

3.1 Return Codes

Every IDAVIE_API function returns idavie_status_t (int32_t). Zero is IDAVIE_OK. Negative values are stable named ABI constants. Values 1–32767 are reserved for CFITSIO pass-through; plug-ins wrapping CFITSIO must negate before returning.

3.2 Thread-Local Message String

Each plug-in exports idavie_last_error_message(void), returning a const char*. Valid only until the next ABI call on the same thread; always non-null. C# callers marshal as IntPtr and copy immediately via Marshal.PtrToStringUTF8.

3.3 Output Parameters

Output parameters are undefined after a non-zero return and must not be read. This prevents the spectralProfileSize 4-vs-8 byte mismatch from producing silent garbage: if GetSourceStats fails, the caller never touches the SourceStats struct.

3.4 Load-Time Failure

NativePluginLoader.cs currently logs missing symbols as warnings and defers failure to a NullReferenceException. Under this ABI, if idavie_abi_version is absent or out of range the loader throws immediately at load time.

3.5 Status Code Reference

Constant	Value	Meaning
IDAVIE_OK	0	Success — output parameters are valid.
IDAVIE_ERR_INVALID_ARGUMENT	1	A required input argument is null or out of range.
IDAVIE_ERR_NULL_HANDLE	2	An opaque handle passed to a function is null.
IDAVIE_ERR_OUT_OF_MEMORY	3	Plug-in could not allocate a required buffer.
IDAVIE_ERR_IO	4	Filesystem or network read/write error.
IDAVIE_ERR_NOT_FOUND	5	A requested resource (file, HDU, source ID) was not found.
IDAVIE_ERR_UNSUPPORTED	6	The operation is not supported by this plug-in version.
IDAVIE_ERR_FORMAT	7	Input data is malformed or unrecognisable.
IDAVIE_ERR_OUT_OF_RANGE	8	A value exceeds the defined range for this parameter.
IDAVIE_ERR_CANCELLED	9	A progress callback returned non-zero; operation was aborted.
IDAVIE_ERR_VERSION_MISMATCH	10	MAJOR version of plug-in does not match the kernel.
IDAVIE_ERR_INTERNAL	99	Unexpected internal error; check idavie_last_error_message.

4 Threading Model

Internal parallelism is a private implementation detail. The public ABI is single-threaded per buffer, so Unity jobs and the render thread coexist safely with plug-in calls.

Rule	Detail
Per-buffer rule	No two calls may run concurrently on the same buffer pointer. Calls on independent buffers may run concurrently. No global mutable state is permitted in the public ABI surface.
Internal parallelism	data_analysis_tool.h includes omp.h but no function documents re-entrancy. Functions may use OpenMP internally but must complete all worker threads before returning and must not expose shared state to callers.
IDAVIE_THREAD_SAFE annotation	Functions marked IDAVIE_THREAD_SAFE must read only from const-qualified inputs and write only to their own output pointers. All others are assumed unsafe for concurrent use.
C# dispatcher	All native delegates are invoked from a single dispatcher thread in the Infrastructure layer. The dispatcher also resolves the UpdateMaskVoxel violation (FitsReader.cs:601) where Vector3Int leaks UnityEngine into the binding: it decomposes the Unity type into three int32_t scalars before crossing the boundary.

5 Memory Ownership

A single idavie_free and a clear allocator-owns rule eliminate cross-DLL heap mismatches and both sizeof corruption bugs identified in the Day 2 baseline.

Rule / Symbol	Detail
idavie_free(idavie_buffer_t buf)	The single free function for all plug-in-owned allocations. C# callers must never call Marshal.FreeHGlobal or Marshal.FreeCoTaskMem on plug-in pointers. Replaces FreeDataAnalysisMemory, FreeFitsPtrMemory, and FreeAstMemory, eliminating cross-DLL heap mismatch on Windows.

Rule / Symbol	Detail
`Caller-allocated buffers`	Native functions must not free caller-allocated input buffers. Where output size is known in advance, the preferred pattern is (T* out, int64_t capacity, int64_t* written).
`By-value structs`	Structs passed by value (SourceInfo, SourceStats) are copied; neither side frees a by-value field. Buffers returned via output pointer are valid until passed to idavie_free.
`Known bug — SaveSubMask`	FitsReader.cs:634 allocates sizeof(int) × N for a buffer read as long* — corrupts on Linux. Fixed by native-side buffer ownership and fixed-width types from Section 2.
`Known bug — SaveNewInt16Mask`	FitsReader.cs:384 allocates sizeof(long) × N for FitsCreateImg, which reads four-byte longs — corrupts on Windows. Fixed by the same rule as above.

6 ABI Compatibility Policy

SemVer in idavie_abi_version plus sizeof assertions on every struct give the kernel a deterministic protocol for loading and evolving plug-ins without silent layout corruption.

6.1 Version Encoding

Rule	Detail
Version encoding	idavie_abi_version() returns a uint32_t: MAJOR in bits 31–16, MINOR in bits 15–0. The kernel rejects any plug-in with a differing MAJOR. A higher plug-in MINOR is permitted; the kernel must not call symbols beyond its own MINOR. A lower plug-in MINOR than required is a load-time error.

6.2 Change Classification

Change type	Examples
No version bump required	Adding new exported symbols at the end of the ABI surface. Adding new IDAVIE_ERR_ constants outside the existing range. Adding IDAVIE_THREAD_SAFE annotations to previously unannotated functions.
MINOR bump required	Adding a new symbol the kernel will call. Changing documented thread-safety of an existing function.
MAJOR bump required (breaking — otherwise forbidden)	Removing or renaming any exported symbol. Changing any function signature. Changing struct field order, size, or type.

6.3 Struct Layout Assertions

Every ABI struct must carry a static_assert in C++ and a Marshal.SizeOf assertion in C# at startup. New fields may only be appended, and only in a new MAJOR version. SourceStats.spectralProfileSize — int64_t in C++, int in C# — is the canonical example of what happens without these checks.

Struct	Size	C++ assertion	C# assertion
`idavie_source_info_t`	**56 bytes**	`static_assert(sizeof(idavie_source_info_t) == 56)`	`Marshal.SizeOf<SourceInfoT>() == 56`

Struct	Size	C++ assertion	C# assertion
`idavie_source_stats_t`	**128 bytes**	`static_assert(sizeof(idavie_source_stats_t) == 128)`	`Marshal.SizeOf<SourceStatsT>() == 128`
`idavie_buffer_t`	**24 bytes**	`static_assert(sizeof(idavie_buffer_t) == 24)`	`Marshal.SizeOf<BufferT>() == 24`

7 Requirements Traceability

All rules in this specification trace to the following sources:

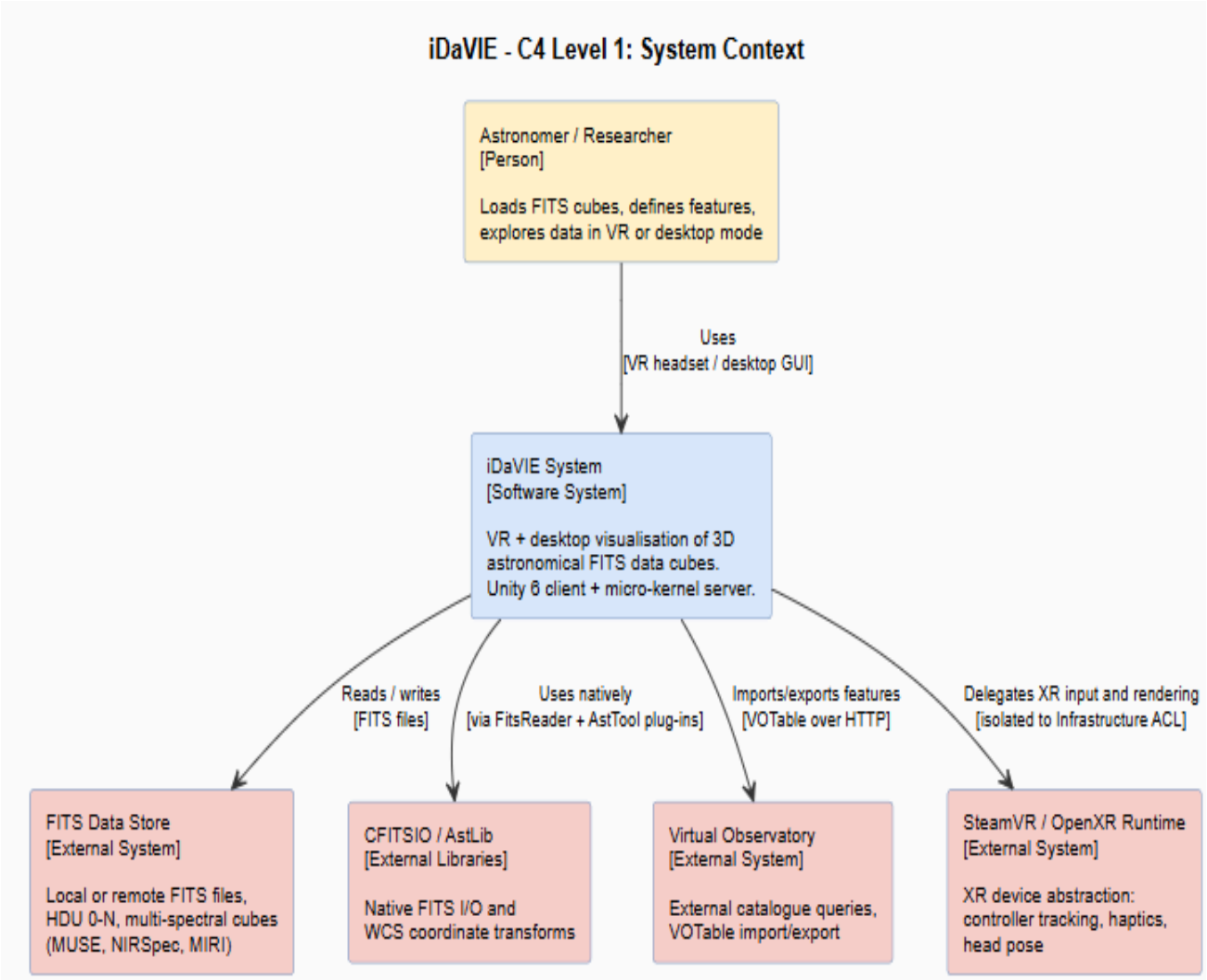
Source	Description	Sections covered
ADR-004	Standardise Plug-in ABI for Native C/C++ Extensions	All sections
ISO/IEC 25010:2023	Modularity, Modifiability sub-characteristics	Sections 2, 5, 6
iDaVIE Assessment Spec §4.2 Constraint 5	Formal plug-in contract with ABI stability, SemVer, isolation, threading, memory ownership	All sections
iDaVIE Assessment Spec §6.1	Architecture and Micro-kernel Core requirements	Sections 3, 4
Day 2 baseline	spectralProfileSize int64_t/int mismatch; SaveSubMask sizeof(int) bug; SaveNewInt16Mask sizeof(long) bug	Sections 2, 5
FitsReader.cs:601	Vector3Int leaking UnityEngine into native binding — dispatcher resolution	Section 4
NativePluginLoader.cs:147	Missing symbol handling — load-time failure hardening	Section 3.4

C4 Diagram

Level 1 — System Context (PlantUML)

Shows iDaVIE as a black box and everything it interacts with.

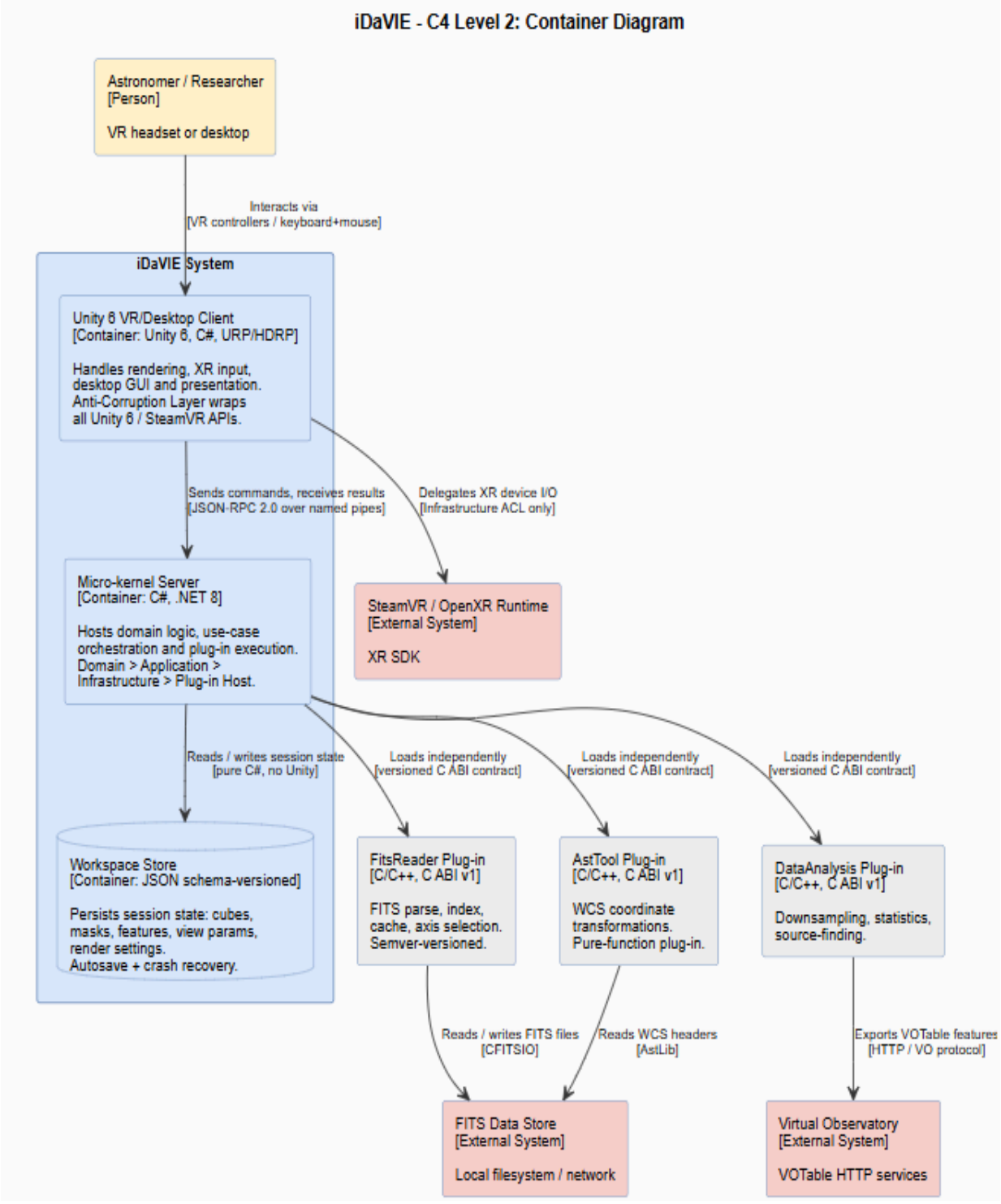
Establishes iDaVIE's boundary and its external dependencies: the astronomer as primary user, FITS files for data input, SteamVR/OpenXR for the VR runtime, CFITSIO/AstLib for native FITS I/O and WCS transforms, and the Virtual Observatory for VOTable exchange. No internal structure is shown.



Level 2 — Container Diagram (PlantUML)

Zooms into iDaVIE and shows the major deployable units.

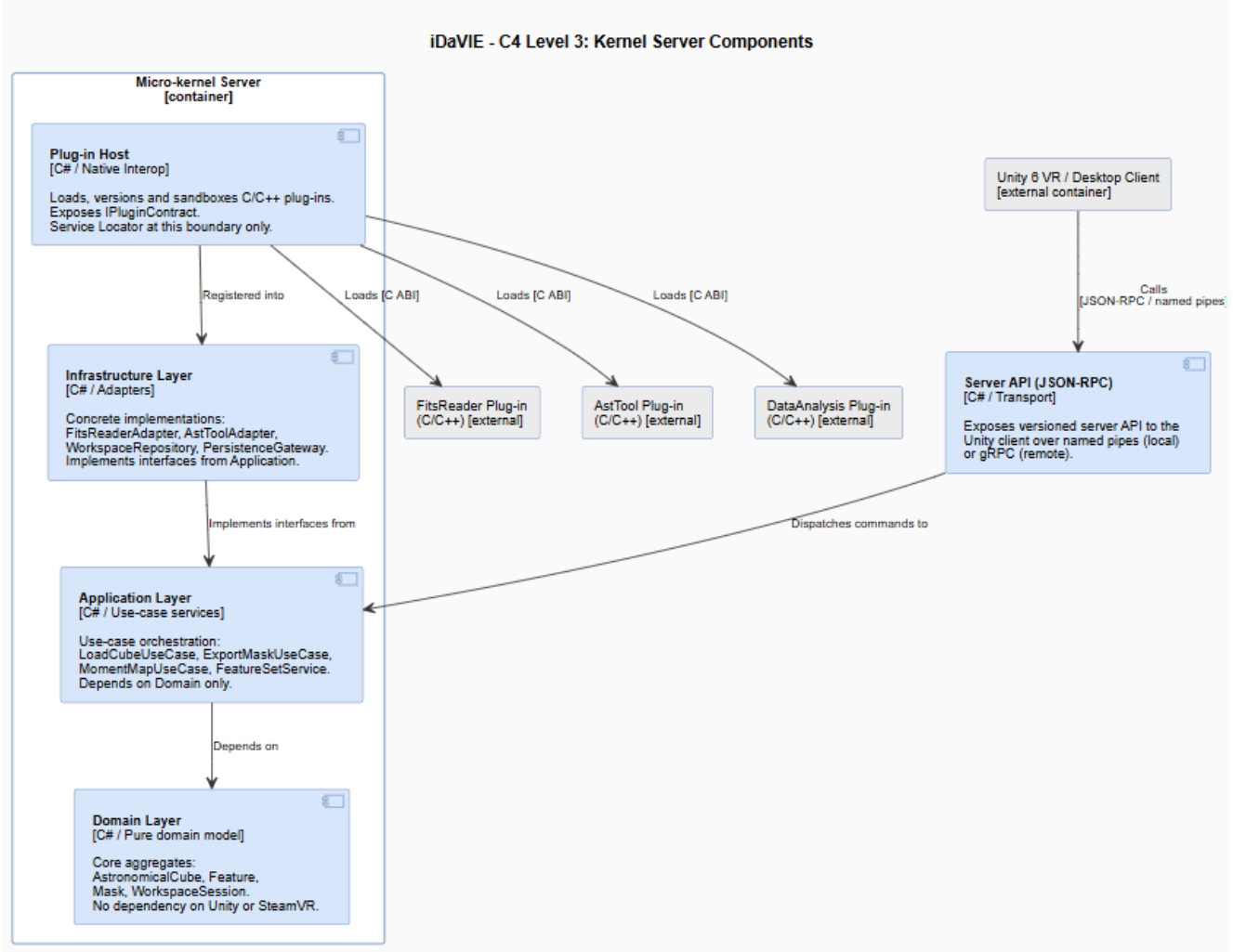
Decomposes iDaVIE into its major deployable units: the Unity 6 Client for rendering and presentation, the Micro-kernel Server for domain logic and plug-in execution, three independently-loadable C/C++ plug-ins (FitsReader, AstTool, DataAnalysis) exposed via a versioned C ABI, and the Workspace Store for session persistence. Client-to-server communication is JSON-RPC over named pipes.



Level 3 — Component Diagram: Micro-kernel Server (PlantUML)

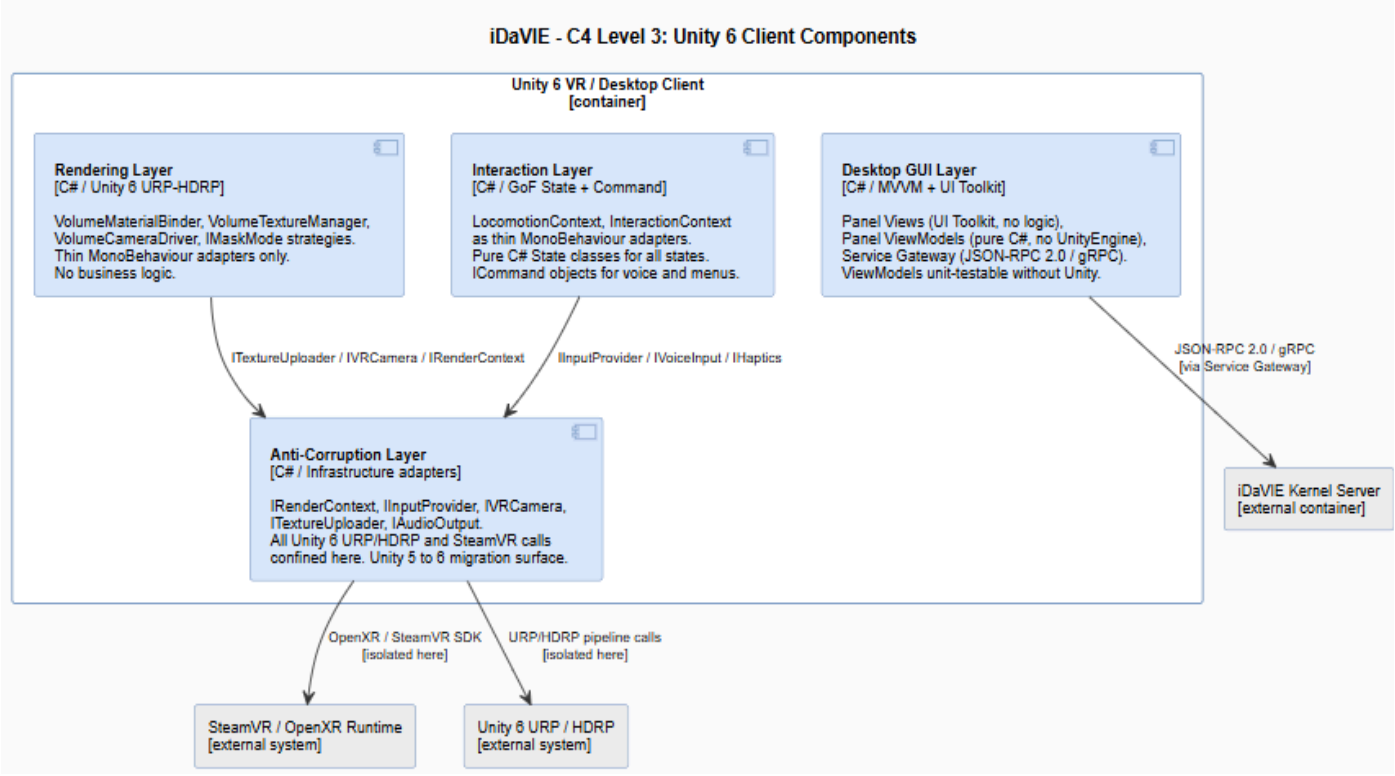
Zooms into the server container and shows the layered internal components.

Shows the server's internal layered structure with a strict downward dependency rule: Domain ← Application ← Infrastructure ← Plug-in Host. The Domain Layer has no Unity or SteamVR imports. The Plug-in Host is the only location where a Service Locator is permitted.



Level 3 — Component Diagram: Unity 6 Client (PlantUML)

Shows the client's internal components. The Anti-Corruption Layer wraps all UnityEngine and SteamVR calls, keeping domain code Unity-free and containing the Unity 5 to Unity 6 migration. The Interaction System and Rendering Layer depend only on interfaces. The Service Gateway is the single point of communication with the server.



Level 4 — Code Diagram (Before/after refactor of VolumeDataSetRenderer)

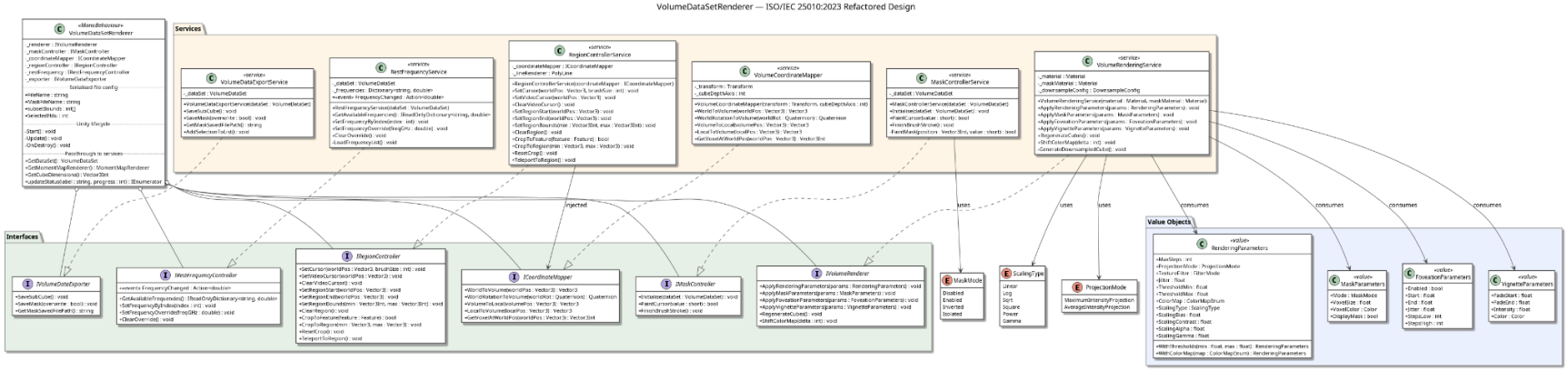
This maps to your **worked refactoring example** deliverable. Show the before state, then the after.

Before/after VolumeDataSetRenderer3, the codebase's primary god class (WMC ~45, CBO ~28). Decomposed into VolumeMaterialBinder, VolumeTextureManager, VolumeCameraDriver and FoveatedSamplingPolicy, each depending only on interfaces. Mask modes extracted as Strategy implementations behind IMaskMode, satisfying OCP.

Before — Monolithic god class

After — Decomposed, SOLID-compliant

VolumeDataSetRenderer — ISO/IEC 25010:2023 Refactored Design



Architecture Decision Records (ADRs) Architecture Decision Record Log

ADR-001: Adopt Layered Micro-kernel Architecture

Status: Proposed

Date	18 May 2026
Deciders	Architecture Guild (All Sub-team Tech Leads)
Learning Outcomes	LO3, LO4
Architectural Drivers	ISO 25010: Modularity, Modifiability, Testability

Context

- The assignment specification mandates a micro-kernel server, layered architecture and strict dependency flow (Domain → Application → Infrastructure → Plug-in Host).
- The current iDaVIE codebase exhibits tightly coupled MonoBehaviour scripts, mixed responsibilities, lifecycle logic embedded inside domain code, and monolithic classes (e.g., VolumeDataSetRenderer).
- An imminent platform migration from Unity 5 to Unity 6 is a strategic driver; the current architecture makes this migration high-risk.

Decision

- Adopt a client–server architecture at the system level: the Unity 6 VR client handles rendering, input and presentation; the server hosts domain logic, plug-in execution and long-running computation.
- Inside the server, apply a micro-kernel pattern: a small, stable kernel exposes a versioned plug-in contract; data formats, domain operations and coordinate systems are realised as C/C++ plug-ins.
- Enforce a strict layered dependency flow inside the kernel: Domain → Application → Infrastructure → Plug-in Host. Downward-only dependencies are mandatory.
- Architecture violations (upward dependencies, circular dependencies) are treated as a CI build failure from Day 3 onwards.

Consequences — Positive

- Directly satisfies the mandatory architectural constraints in the specification (Section 4.2).
- Reduces coupling between systems, improving both Modularity and Modifiability (ISO 25010).
- Enables independent testing of each layer without Unity or SteamVR present.
- Isolates Unity 5→6 migration surface to the Infrastructure and Plug-in Host layers only.
- Enables future extensibility (subcube loading, HDU selection, multiplayer) as plug-in additions, not core rewrites.

Consequences — Negative

- Introduces additional architectural complexity and indirection.
- Requires more interfaces and abstractions, increasing initial design effort.
- Minor performance overhead due to abstraction layers; acceptable given the 90 fps floor is GPU-bound, not logic-bound.
- Onboarding cost for team members unfamiliar with micro-kernel patterns.

Alternatives Considered

- Retain existing monolithic Unity MonoBehaviour architecture — rejected: violates the specification and entrenches all known maintainability defects.
- Traditional layered client-only architecture without a micro-kernel — rejected: does not support plug-in extensibility or ABI versioning required by the spec.
- Bi-directional dependencies between layers — rejected: immediately creates cyclic dependencies which are a fail condition per Section 4.2.

Notes & Traceability

This ADR is the root architectural decision; all other ADRs trace back to it. CK metric improvement targets (WMC ≤ 20, CBO ≤ 14, LCOM ≤ 0.5) are enforced in CI as fitness functions.

ADR-002: Introduce Anti-Corruption Layer Around Unity and SteamVR

Status: Proposed

Date	18 May 2026
Deciders	Architecture Guild
Learning Outcomes	LO3, LO4, LO6
Architectural Drivers	ISO 25010: Testability, Modifiability; Spec Section 4.2 Constraint 3

Context

- The specification explicitly requires that domain code must not transitively depend on UnityEngine or SteamVR (Section 4.2, Constraint 3).
- The current implementation tightly couples business logic — FITS rendering math, feature statistics, coordinate transformations — directly to Unity-specific APIs and MonoBehaviour lifecycles.
- This coupling makes unit testing outside Unity impossible, and means that any Unity 5→6 API change ripples through the entire codebase.

Decision

- Introduce an Anti-Corruption Layer (ACL) as a set of stable C# interfaces that domain and application code depend on, with concrete adapter implementations in the Infrastructure layer.
- Define the following core abstractions (minimum set): IRenderContext (render pipeline operations), IInputProvider (XR input, replaces direct SteamVR calls), IVRCamera (camera and foveation data), ITextureUploader (GPU texture management), IAudioOutput (spatial audio).
- Unity MonoBehaviours and SteamVR SDK calls are isolated to concrete adapter classes in the Infrastructure layer only.
- Domain and Application layers import only these interfaces — zero direct UnityEngine or SteamVR usings are permitted above the Infrastructure layer. This is enforced by NDepend CQLinq rules.

Consequences — Positive

- Makes pure C# unit testing of domain and application logic possible without a Unity runtime.
- Confines the Unity 5→6 migration to adapter implementations; domain code is untouched.
- Allows domain logic (FITS math, feature statistics) to be reused outside Unity if required.
- Enables mock/stub injection for all platform APIs in test doubles.
- Reduces Mocking Difficulty metric — a key Testability indicator.

Consequences — Negative

- Additional adapter classes increase codebase size and maintenance surface.
- Developers must keep abstraction interfaces current as Unity 6 APIs evolve.
- Risk of leaky abstraction if adapter boundaries are not enforced via CI rules.

Alternatives Considered

- Direct UnityEngine usage throughout the codebase — rejected: violates Constraint 3 and prevents any offline unit testing.
- Wrapper utility classes without formal interfaces — rejected: does not enforce the boundary and cannot be mocked.
- Complete lock-in to Unity XR Toolkit — rejected: over-constrains future hardware migration and contradicts Modifiability goals.

Notes & Traceability

Every public interface in the ACL must be covered by at least one test double (Constraint 4). Interface members should be kept to ≤ 7 (ISP target in Section 7.2 Testability). SteamVR-specific dependencies in LocomotionState and InteractionState are the primary migration targets.

ADR-003: Replace Singleton-Based Services with Dependency Injection

Status: Proposed

Date	18 May 2026
Deciders	Architecture Guild
Learning Outcomes	LO4, LO6
Architectural Drivers	ISO 25010: Testability, Modularity; SOLID: DIP, SRP

Context

- The specification explicitly identifies heavy reliance on singleton services as a known maintainability pressure (Section 1.1).
- Singletons create hidden dependencies (high CBO), prevent dependency mocking, and couple subsystems that should be independent.
- Current patterns such as direct static access (e.g., VolumeDataSetRenderer.Instance) make Mocking Difficulty high and branch coverage impossible for caller classes.
- The Scrum-of-Scrums team structure requires sub-teams to work on parallel components without runtime coupling; singletons undermine this independence.

Decision

- Replace all singleton access patterns with constructor injection at the class level.
- Introduce explicit interfaces (per ADR-002) at every service boundary.
- Restrict service registration to a single composition root per application/server entry point — one for the server kernel, one for the Unity client shell.
- The plug-in registry at the kernel boundary may use a Service Locator pattern (GRASP: Indirection), as this is an accepted exception for dynamic plug-in discovery. Everywhere else uses constructor injection.
- Static utility classes that carry state are prohibited. Pure static helpers with no state are acceptable.

Consequences — Positive

- Testability improves immediately: any class can be unit-tested by injecting mocks.
- Reduces hidden coupling — CBO values for previously singleton-dependent classes are expected to drop significantly.
- Supports SOLID (DIP) and GRASP (Low Coupling, Protected Variations) principles.
- Sub-teams can integrate via interface contracts without sharing live singleton instances.
- Baseline vs. projected CK metrics for CBO and LCOM are expected to show the largest improvements here.

Consequences — Negative

- Increased setup complexity at composition roots.
- Developers must manage DI configuration, which requires additional documentation.
- Initial refactoring of existing singletons is non-trivial — scope must be managed per sprint.

Alternatives Considered

- Continue using global singleton services — rejected: directly prevents testability goals and is flagged by the specification.
- Use static utility classes — rejected: equivalent to singletons from a testability perspective.
- Partial Service Locator throughout — rejected: Service Locator is permitted only at the kernel plug-in boundary; general use hides dependencies.

Notes & Traceability

The composition root for the server kernel is owned by Sub-team 1 (Architecture). Each sub-team owns the DI wiring for its own components. The Quality Guild enforces 'no new singleton introductions' via SonarQube custom rules from Day 3.

ADR-004: Standardise Plug-in ABI for Native C/C++ Extensions

Status: Proposed

Date	18 May 2026
Deciders	Architecture Guild, Data I/O Sub-team
Learning Outcomes	LO3, LO5
Architectural Drivers	ISO 25010: Modularity, Reusability; Spec Section 4.2 Constraint 5

Context

- The specification requires a formal plug-in contract with ABI stability, semantic versioning, plug-in failure isolation, defined threading guarantees, and explicit memory ownership rules (Section 4.2, Constraint 5).
- The current native DLLs (FitsReader, DataAnalysis, AstTool) lack formalised interoperability standards; they are called directly with no versioning, no isolation, and implicit memory ownership.
- A failed or misbehaving native plug-in currently has the potential to crash the entire Unity process.
- Upcoming requirements (subcube loading, HDU selection, MUSE/NIRSpec/MIRI compatibility) will require new plug-ins; a stable ABI prevents regression.

Decision

- Define a stable C ABI boundary for all native plug-ins using plain C function signatures only (no C++ name mangling across the boundary).
- Apply semantic versioning to each plug-in: MAJOR.MINOR.PATCH. ABI must remain compatible within a major version.
- Use opaque handles (e.g., idavie_fits_handle_t*) for all heap-allocated objects crossing the boundary. Ownership rules (caller-allocated vs. callee-allocated, explicit free functions) must be documented in the header.
- Define explicit threading rules per plug-in: whether calls must occur on a specific thread, whether re-entrant calls are safe, and whether callbacks fire on background threads.
- Each plug-in exposes a standard probe function (idavie_plugin_info) returning version, capabilities, and ABI level, enabling the kernel to perform compatibility checks at load time.
- Plug-in failures are caught at the kernel boundary and surfaced as structured error codes, never as unhandled exceptions propagating into the kernel.

Consequences — Positive

- Long-term plug-in compatibility is guaranteed across releases, fulfilling Constraint 5.
- Plug-in failure isolation prevents a broken data plug-in from crashing the VR session.
- New data format plug-ins (MUSE, particle datasets, TIFF) can be added without kernel changes.

- C ABI is the most portable boundary — works from C, C++, Rust, or any language with C FFI.
- Explicit ownership rules eliminate an entire class of memory safety bugs at the native boundary.

Consequences — Negative

- ABI compatibility constraints reduce implementation freedom for plug-in developers.
- More boilerplate per plug-in (probe function, version struct, error enum).
- The versioning policy requires disciplined governance — breaking ABI must trigger a major version bump.

Alternatives Considered

- Direct DLL calls without a formal ABI contract — rejected: current state; is a known fragility and violates the specification.
- C++ virtual interfaces across the boundary — rejected: name-mangling and vtable layout are compiler-specific and ABI-unstable.
- Avoid semantic versioning — rejected: Constraint 5 is explicit; unversioned plug-ins cannot guarantee compatibility.

Notes & Traceability

The ABI header files are a Sub-team 1 deliverable (plug-in ABI specification). Sub-team 2 provides reference bindings for FitsReader, AstTool and DataAnalysis. Contract tests must be runnable without Unity (Sub-team 1 testing deliverable).

ADR-005: Enforce Architecture Rules in CI

Status: Proposed

Date	19 May 2026
Deciders	Quality Guild, Architecture Guild
Learning Outcomes	LO1, LO6, LO7
Architectural Drivers	ISO 25010: Modularity, Analysability; Spec Section 4.2

Context

- Manual architecture enforcement is unreliable at scale across a 28-person Scrum-of-Scrums team with 7 parallel sub-teams.
- The specification mandates architecture validation and maintainability tooling (Sections 7.3 and 10.3); the CI pipeline must block PRs on violations.
- Architecture decay (dependency cycles, upward layer references, singleton reintroduction) can silently accumulate across sprints if not automatically caught.
- The Quality Guild owns the CI pipeline collectively; no single sub-team is responsible, so rules must be codified and machine-enforced.

Decision

- Configure the GitHub Actions CI pipeline to run the full metric and architecture rule suite on every pull request. PRs failing any gate are blocked from merge.
- Integrate the following tools in CI by end of Sprint 1 (Day 5): SonarQube Cloud (complexity, duplication, smells, coverage), NDepend (CQLinq layer rules, instability, propagation cost), DV8 (Dependency Structure Matrix, architectural anti-patterns).
- Define hard quality gates: (a) zero circular dependencies at namespace and assembly level; (b) zero architecture violations against the layer rule; (c) CK metrics within spec thresholds (WMC ≤ 20 domain, CBO ≤ 14 domain, LCOM ≤ 0.5, RFC ≤ 50, DIT ≤ 4); (d) branch coverage ≥ 70% on domain layer; (e) no new UnityEngine usings above Infrastructure layer.
- The CI pipeline posts a metric dashboard delta as a PR comment, showing before/after values for all CK metrics on changed files.
- Gate hardening schedule: basic syntax checks by Day 1 end; full metric suite on every PR by Day 5; full architecture violation blocking by Day 10.

Consequences — Positive

- Architectural decay is caught at PR time, not post-merge.
- Provides objective evidence for the worked refactoring examples (before/after CK metrics are CI-generated).
- Enforces consistent quality standards across all 7 sub-teams without relying on code review.
- CI dashboard is a live deliverable for the pitch (T6).

Consequences — Negative

- Stricter gates may slow merges, particularly early in Sprint 1 when the baseline is far from targets.
- False positives in NDepend/DV8 may require rule tuning; the Architecture Guild owns this.
- CI infrastructure must be maintained alongside the proposal work.

Alternatives Considered

- Manual code review only — rejected: insufficient at 28-person scale with 7 parallel sub-teams.
- Optional architecture validation — rejected: specification requires blocking gates.
- Post-merge quality checking — rejected: allows decay to accumulate; contradicts the intent of the specification.

Notes & Traceability

The Quality Guild is collectively accountable for CI. NDepend CQLinq rules for the layer boundary are authored by Sub-team 1 (Architecture). All gate thresholds are drawn directly from Section 7.1 (CK suite) and Section 7.2 (metric families).

ADR-006: Separate Domain Logic from MonoBehaviour Lifecycle

Status: Proposed

Date	19 May 2026
Deciders	Architecture Guild
Learning Outcomes	LO4, LO5, LO6
Architectural Drivers	ISO 25010: Testability, Modifiability; SOLID: SRP

Context

- The current architecture intermixes Unity lifecycle methods (Awake, Start, Update, OnDestroy) directly with business logic — feature statistics calculation, FITS data loading, coordinate transforms — inside MonoBehaviour classes.
- MonoBehaviour classes cannot be instantiated outside a Unity runtime, making domain logic untestable in edit-mode or headless CI.
- Unity lifecycle coupling is one of the specification's explicitly named maintainability pressures (Section 1.1), and violates the Single Responsibility Principle.
- Unity 6 introduces significant lifecycle changes (Awake/Start ordering, managed code stripping); mixing domain logic increases migration risk.

Decision

- Restrict MonoBehaviours to thin adapters, controllers and presenters only. Their sole responsibilities are: subscribing to Unity lifecycle events, delegating to domain/application services, and binding data to Unity GameObjects.
- Move all business logic into pure C# classes (POCOs) in the Domain and Application layers, with no MonoBehaviour inheritance and no UnityEngine usings.
- The MonoBehaviour adapter calls into domain services via injected interfaces (per ADR-003). It never contains conditional logic beyond trivial null-guards.
- This pattern applies across all sub-teams: VolumeDataSetRenderer adapter → VolumeMaterialBinder/VolumeTextureManager domain services; LocomotionState adapter → pure C# State machine; CanvassDesktop adapter → ViewModel classes.

Consequences — Positive

- Domain and Application layer classes become fully unit-testable in edit-mode and headless CI.
- WMC and RFC metrics for the now-split classes are expected to fall well within CK thresholds.
- Unity 6 lifecycle changes only affect the thin adapter layer; domain logic is immune.
- Supports parallel sub-team development: domain logic can be developed and tested without Unity Editor.

Consequences — Negative

- Requires additional service abstraction classes per sub-team.
- Developers must manage synchronisation between Unity's main thread and potentially background domain services.
- More files and namespaces — manageable with the namespace rules in ADR-008.

Alternatives Considered

- Keep business logic inside MonoBehaviours — rejected: prevents testability and violates SRP; directly contradicted by the specification.
- Partial extraction only for selected systems — rejected: creates inconsistency; the CI rules in ADR-005 enforce this globally anyway.

Notes & Traceability

This ADR is the direct mechanism by which Mocking Difficulty is reduced and branch coverage targets (≥ 70% domain) become achievable. Each sub-team's worked refactoring examples must demonstrate this split with before/after CK deltas.

ADR-007: Adopt Event-Driven Communication for Cross-System Integration

Status: Proposed

Date	20 May 2026
Deciders	Architecture Guild
Learning Outcomes	LO3, LO4
Architectural Drivers	ISO 25010: Modularity, Modifiability; GRASP: Low Coupling, Indirection

Context

- The current architecture relies heavily on direct object references between subsystems (e.g., VolumeDataSetRenderer directly calling FeatureSetManager, interaction scripts directly referencing the renderer).
- Direct references create transitive coupling (high CBO), make sub-team integration at interface boundaries difficult, and propagate changes widely.
- Sub-teams work on isolated behavioural packages; they need an integration spine that does not create compile-time dependencies between packages.
- The specification requires plug-in friendliness; plug-ins must not be able to hold direct references into the kernel's domain objects.

Decision

- Adopt domain events and a message bus (event aggregator) for all cross-system communication within the kernel.
- Define typed domain event classes (e.g., FitsDataLoadedEvent, FeatureCreatedEvent, MaskAppliedEvent, ViewStateChangedEvent) in the Domain layer. These are pure C# value objects with no dependencies.
- Sub-systems publish events to the bus and subscribe to events from other systems; they never hold direct references to other sub-system classes.
- The event bus interface (IEventBus) lives in the Application layer. The concrete implementation lives in Infrastructure.
- Unity-side communication between client components (rendering ↔ interaction ↔ GUI) uses Unity's existing event system (UnityEvent / C# events on the adapter layer) — the domain event bus is server/kernel-side only.
- Synchronous direct calls are still used within a single bounded context (e.g., within the Feature domain itself); events are used at bounded context boundaries.

Consequences — Positive

- Eliminates direct cross-package references; sub-teams integrate via event contracts, not shared object graphs.
- CBO for orchestrator classes is expected to drop significantly.
- Plug-ins can subscribe to and publish domain events without kernel source access.
- Extensibility: new sub-systems can observe existing events without modifying publishers (Open-Closed).

- Improves independent testability: any sub-system can be tested by firing events against a mock bus.

Consequences — Negative

- Event tracing and debugging become more difficult; event flow is implicit rather than explicit.
- Implicit control flow reduces analysability — mitigated by structured event logging (observable event bus in dev mode).
- Event ordering guarantees must be specified and tested; undefined ordering can introduce subtle bugs.

Alternatives Considered

- Continue using direct object references — rejected: maintains the coupling that the specification identifies as a core maintainability defect.
- Centralised singleton coordinator/manager — rejected: equivalent to current state and reintroduces the singleton problem addressed in ADR-003.
- Shared mutable state via a global state object — rejected: thread-unsafe and severely limits testability.

Notes & Traceability

Domain event types are a shared contract: Sub-team 1 defines the event taxonomy; sub-teams agree at Architecture Guild before Sprint 2 begins. Event schema changes follow the same semantic versioning rules as the plug-in ABI.

ADR-008: Define and Enforce Package and Namespace Dependency Rules

Status: Proposed

Date	20 May 2026
Deciders	Architecture Guild
Learning Outcomes	LO3, LO7
Architectural Drivers	ISO 25010: Modularity; Spec Section 4.2 Constraint 2

Context

- The specification explicitly forbids circular dependencies between top-level components (Section 4.2, Constraint 2; cyclic package dependencies are a fail condition).
- With 7 sub-teams working concurrently, namespace boundaries are the first line of defence against unintended coupling.
- The current project structure does not enforce dependency ownership — any script can reference any other script, enabling accidental coupling between unrelated systems.

Decision

- Define the following top-level C# namespace hierarchy, one per architectural layer and sub-system: iDaVIE.Domain.{Feature, Rendering, Interaction, Persistence, DataIO}; iDaVIE.Application.{Feature, Rendering, Interaction, Persistence, DataIO, GUI}; iDaVIE.Infrastructure.{Unity, SteamVR, NativePlugins, Persistence}; iDaVIE.Client.{Shell, VR, Desktop}.
- Enforce these dependency rules in NDepend CQLinq (and ArchUnit-equivalent): Domain namespaces must not reference iDaVIE.Infrastructure, iDaVIE.Client, UnityEngine, or SteamVR. Application namespaces must not reference iDaVIE.Client. Infrastructure must not reference iDaVIE.Domain directly (only via interfaces). Plug-ins must communicate exclusively through the ABI contract (ADR-004) and IEventBus (ADR-007).
- Dependency cycles at any level (namespace or assembly) are a CI block gate (ADR-005).
- All refactoring worked examples must show the before/after namespace mapping alongside the CK metrics.

Consequences — Positive

- Eliminates circular dependencies — the fail condition in the specification is structurally prevented.
- Package Instability (I = Ce / (Ca+Ce)) can be measured and managed per the Stable Dependencies Principle.
- Sub-team integration contracts are namespace boundaries; coupling between sub-teams is visible in the DSM.

- Architectural governance is automatable with the tools already in use (NDepend, DV8).

Consequences — Negative

- Requires continuous CI enforcement — rules must be maintained as the architecture evolves.
- May slow rapid prototyping in Sprint 1; the gate hardening schedule in ADR-005 mitigates this by allowing warnings before hard blocks.
- Namespace refactoring of existing iDaVIE classes is a non-trivial initial cost.

Alternatives Considered

- Allow unrestricted namespace references — rejected: fails the specification's fail condition immediately.
- Rely on developer discipline only — rejected: insufficient at 28-person scale.
- Informal dependency conventions without tooling — rejected: untestable and unenforceable.

Notes & Traceability

The namespace map is a Sub-team 1 deliverable; published by end of Day 3 so all sub-teams can scaffold correctly. The DSM at each sprint boundary (DV8) provides evidence of adherence for the Integration & Metrics Report.

ADR-009: Adopt MVVM for the Desktop GUI Client Shell

Status: Proposed

Date	21 May 2026
Deciders	Desktop GUI Sub-team, Architecture Guild
Learning Outcomes	LO4, LO5
Architectural Drivers	ISO 25010: Testability, Modularity; SOLID: SRP, DIP

Context

- The current CanvassDesktop class is a monolithic God-canvas responsible for menu structure, panel state, file dialogs, configuration, and direct calls to native plug-ins and Unity APIs.
- This violates SRP, creates high WMC and CBO, and makes any ViewModel logic untestable without the Unity Editor.
- The specification explicitly requires an MVVM-style split for the Desktop GUI sub-team (Section 6.6) and specifies a client–server transport contract (JSON-RPC over named pipes / gRPC).
- Unity 6 UI Toolkit is the target presentation layer; it supports data binding patterns that make MVVM natural.

Decision

- Decompose CanvassDesktop into the MVVM triad: View (Unity 6 UI Toolkit, no business logic), ViewModel (pure C#, owns presentation state and commands, no UnityEngine dependency), Service Gateway (translates ViewModel commands to server calls via the transport contract).
- Each panel (File, Render, Stats, Sources, Debug) is a separate View/ViewModel pair. No single God-canvas.
- Commands in the ViewModel follow the Command pattern (reified, replayable, testable) — consistent with the Interaction sub-team's approach (ADR-010).
- The client–server transport is specified as JSON-RPC 2.0 over named pipes for local mode, with a gRPC upgrade path for future remote streaming.
- ViewModels have no UnityEngine imports; they are testable with standard xUnit/NUnit without Unity runtime.

Consequences — Positive

- ViewModel unit tests require no Unity; directly achieves the ≥ 70% branch coverage target on this layer.
- Clear SRP: View handles layout, ViewModel handles state, Service Gateway handles transport.
- Composable panels with explicit contracts prevent re-emergence of the God-canvas.
- Transport contract (JSON-RPC/gRPC) is a versioned API; the GUI sub-team and server sub-teams integrate via the contract, not shared objects.

Consequences — Negative

- MVVM data binding in Unity UI Toolkit is less mature than in WPF/React; patterns must be established early.
- JSON-RPC serialisation adds latency for high-frequency interactions; mitigation is batching or local in-process shortcuts for performance-critical calls.
- More files and classes per panel; complexity is managed through the namespace rules in ADR-008.

Alternatives Considered

- Retain monolithic CanvassDesktop with minor decomposition — rejected: does not meet SRP requirements or testability targets.
- MVC instead of MVVM — rejected: MVC Controller still has Unity dependencies; MVVM ViewModel is Unity-free by design.
- Direct binding to server via REST — rejected: JSON-RPC is lower overhead for local IPC and is transport-agnostic.

Notes & Traceability

The transport contract (JSON-RPC schema) is a shared deliverable between Sub-team 6 (Desktop GUI) and Sub-team 1 (Architecture). It must be agreed by Day 6 to unblock parallel development.

ADR-010: Formalise State and Command Patterns for the Interaction System

Status: Proposed

Date	21 May 2026
Deciders	Interaction Sub-team, Architecture Guild
Learning Outcomes	LO4, LO5
Architectural Drivers	ISO 25010: Testability, Analysability; SOLID: OCP, ISP

Context

- The VR Interaction System currently uses two implicit state machines (LocomotionState, InteractionState) implemented as large switch statements or flag-based conditionals inside MonoBehaviour Update() methods.
- Implicit state machines have high cyclomatic complexity (violating Analysability), are difficult to test (no discrete State objects to instantiate), and are fragile under extension (violating OCP).
- The specification requires that state machines be re-platformed onto Unity 6 Input System with SteamVR replaced by IInputProvider (Section 6.4).
- Voice commands and quick-menu actions currently lack reification; they cannot be replayed, logged, or unit-tested.

Decision

- Implement the GoF State pattern for both LocomotionState and InteractionState: extract each named state (Idle, Moving, ScalingRotating, ParameterEditing, CreatingSelection, EditingRegion, SourceEditing, PaintingStroke) as a distinct C# class implementing ILocomotionState / IInteractionState.
- State classes are pure C# POCOs (no MonoBehaviour inheritance); they receive dependencies via constructor injection (ADR-003), including IInputProvider and IVoiceRecogniser.
- The Context class (thin MonoBehaviour adapter) owns the current state reference and delegates all input handling to it.
- Implement the GoF Command pattern for voice commands and quick-menu actions: each action is a reified ICommand object with Execute() and (where applicable) Undo(). Commands are logged, replayable, and unit-testable in isolation.
- Apply Interface Segregation: split SteamVR's monolithic input API into IPointerInput, IGripInput, IVoiceInput, IHaptics — each with ≤ 7 members.

Consequences — Positive

- Cyclomatic complexity per method falls sharply — each State class handles exactly one state's transitions.
- Pure C# State and Command classes are fully unit-testable with mock IInputProvider and IVoiceRecogniser.

- New states or commands are additions (new classes), not modifications — OCP satisfied.
- ISP-compliant interfaces reduce unnecessary coupling; a class needing only haptics does not depend on pointer input.
- Command log enables session replay, accessibility audit, and scenario testing.

Consequences — Negative

- State pattern increases class count significantly (one class per state per machine).
- Initial migration of implicit switch logic to explicit State classes is a substantial refactoring effort — prioritised for Sprint 2.
- Context class must safely manage state transitions; thread-safety on state change must be specified.

Alternatives Considered

- Retain switch-based state machines — rejected: high cyclomatic complexity, untestable, fragile under extension.
- Unity Animator as state machine — rejected: tightly couples state logic to Unity; incompatible with pure C# testing requirement.
- Statecharts/SCXML library — rejected: introduces an unvetted dependency; the GoF State pattern is sufficient and familiar to the team.

Notes & Traceability

UML state machine diagrams for both state machines are a Sub-team 4 deliverable. The IInputProvider interface replaces the direct SteamVR dependency; the concrete SteamVR adapter is an Infrastructure class (consistent with ADR-002).

ADR-011: Define Feature as a First-Class Domain Aggregate

Status: Proposed

Date	22 May 2026
Deciders	Feature System Sub-team, Architecture Guild
Learning Outcomes	LO4, LO5
Architectural Drivers	ISO 25010: Modularity, Testability; GRASP: Information Expert; SOLID: LSP

Context

- The current Feature system mixes identity, statistics, persistence, and visualisation responsibilities inside FeatureSetManager, violating SRP and producing a high-WMC, high-CBO class.
- Feature statistics (voxel count, flux, centroid, W20) are computed outside the Feature entity, violating GRASP Information Expert.
- The three Feature flavours (Masked, Imported, User-defined) are handled by conditionals rather than polymorphism, violating LSP and OCP.
- The Feature domain needs to be server-side (kernel domain layer) and completely free of Unity types to enable moment-map generation and VOTable export without a VR session.

Decision

- Promote Feature to a first-class Domain Aggregate with a well-defined identity, invariants, and a clear aggregate boundary.
- The Feature aggregate encapsulates its own statistics (GRASP Information Expert): voxel count, total/peak flux, flux-weighted centroid, W20, W50 are methods on the aggregate, computed from its internal voxel collection.
- Define an IFeature interface; Masked, Imported, and User-defined Feature are substitutable implementations (Liskov Substitution Principle).
- Decompose FeatureSetManager into: FeatureCatalog (identity and persistence, maps to Sub-team 7 Persistence contracts), FeatureSetService (use-case orchestration, Application layer), FeatureVisualiser (Unity-side adapter, Infrastructure/Client layer).
- Moment-map generation and VOTable export are Application-layer use cases that call into the Feature aggregate via FeatureSetService — no direct Unity dependency.

Consequences — Positive

- LCOM drops significantly: the Feature aggregate is cohesive around its data.
- CBO for FeatureSetManager analogues drops: each split class has a single responsibility.
- Statistics are co-located with data — correct, testable, and not duplicable.
- New Feature types (iso-contours, particle datasets) are new IFeature implementations, not conditional branches.

- VOTable export and moment-map generation are testable in headless CI without Unity.

Consequences — Negative

- Splitting FeatureSetManager requires careful state migration to avoid regressions.
- Aggregate boundary must be agreed with Sub-team 7 (Persistence) by Sprint 2 Day 8 for workspace state contracts.

Alternatives Considered

- Keep FeatureSetManager monolithic — rejected: violates SRP, produces unacceptable CK metric values.
- Compute statistics in a separate stateless service — rejected: violates GRASP Information Expert; the Feature aggregate owns its own data.
- Use inheritance for Feature flavours — rejected: conditional polymorphism via if/switch on type; LSP requires interface-based substitution.

Notes & Traceability

The Feature aggregate UML class diagram and invariants list are a Sub-team 5 deliverable. The state contracts between Feature, Persistence (Sub-team 7), and Data I/O (Sub-team 2) must be signed off at Architecture Guild by Day 9 (Sprint 2 exit criterion).

ADR-012: Establish a Versioned Workspace Persistence Contract

Status: Proposed

Date	22 May 2026
Deciders	Persistence Sub-team, Architecture Guild
Learning Outcomes	LO4, LO5, LO6
Architectural Drivers	ISO 25010: Modifiability, Testability; Spec Section 6.7

Context

- iDaVIE sessions contain significant user state: loaded cubes, mask edits, paint strokes, defined features, view parameters, render settings, selection boxes.
- There is currently no formal persistence boundary; workspace state is implicitly held in MonoBehaviour fields and Unity scene state, making durable save/restore, crash recovery, and forward-compatible versioning impossible.
- The specification (Section 6.7) requires a formal domain model for workspace, a persistence format with schema versioning and migrations, autosave with transactional semantics, and a pure-C# persistence layer with no Unity dependency.
- Sub-team 7 has the smallest team (3 persons in Team Beta) and must deliver state contracts to all other sub-teams by Day 9.

Decision

- Define a Workspace as a domain aggregate: a versioned, serialisable snapshot of all sub-system states. Each sub-team owns a WorkspaceSlice interface declaring what its state looks like (e.g., IRenderState, IFeatureState, IInteractionState).
- Adopt a schema-versioned JSON format (SCHEMA_VERSION field, semver) for the workspace file. Migrations are explicit functions: Migrate_v1_to_v2(), etc.
- Autosave cadence: every 5 minutes and on significant state transitions (mask applied, feature created). Autosave writes to a shadow file; promotion to main save is atomic (write-then-rename).
- Partial and corrupted state is handled gracefully: unknown fields are ignored (forward-compatible), missing required fields fall back to defaults, version mismatch triggers the migration chain.
- The persistence layer is pure C# with no UnityEngine dependency. Unity-specific serialisation (ScriptableObject, PlayerPrefs) is prohibited.
- Round-trip determinism tests are required: save → load → save must produce byte-identical output for deterministic state.

Consequences — Positive

- Crash recovery becomes possible with the shadow-file autosave pattern.
- Forward-compatible versioning allows workspace files to survive application upgrades.
- Pure C# persistence layer is fully unit-testable without Unity.

- Clear WorkspaceSlice contracts enable all sub-teams to declare their state independently, meeting the Day 9 delivery gate.

Consequences — Negative

- JSON schema evolution requires disciplined migration chain maintenance.
- Atomic write-then-rename requires filesystem assumptions; documented and tested on Windows (Unity's primary platform).
- Team Beta Persistence's reduced scope (3 persons) means one worked refactoring example rather than two; the domain model and contracts are full scope.

Alternatives Considered

- Unity PlayerPrefs or ScriptableObject serialisation — rejected: Unity-bound, not testable headlessly, not forward-compatible.
- Binary serialisation — rejected: fragile across .NET versions, not human-readable for debugging, difficult to migrate.
- No formal autosave (manual save only) — rejected: user data loss risk; specification requires crash recovery.

Notes & Traceability

WorkspaceSlice contracts from all sub-teams are mandatory for Team Beta by Day 9 (Architecture Guild sign-off criterion per Section 8.2). Sub-team 7 owns the Workspace aggregate; each sub-team owns its own WorkspaceSlice implementation.

CI rules

detect-changes - runs first, tells every other job what files changed. It means jobs only trigger when relevant files are touched rather than everything running on every push.

human-review-required - intentional failure gate. Any diagram or doc change = PR blocked until a human approves.

build-unity - skips gracefully if no Unity license secret is set. Means the pipeline won't explode for teammates who haven't set up Unity secrets yet.

unity-tests - runs both EditMode and PlayMode but uses continue-on-error so if one fails the other still runs.

build-dotnet - the critical fail-fast gate. Everything else waits for this. If the C# skeletons don't compile, nothing else bothers running.

arch-tests, ck-metrics, circular-deps - three hard blocking gates on code quality.

sonar - skips if no token rather than failing. Means day one before secrets are set up it won't break everything.

diagram-validation - warnings only, never blocks. Correct since the human gate handles the actual blocking.

sprint-metrics - only runs on pushes to main, commits JSON history. Clean.

pr-comment - unified single comment, updates itself rather than spamming new comments every push.

CI Pipeline Test Cases

Five CI gates are defined. Gates 1–4 block merge on failure. Gate 5 is advisory only.

Gate 1: arch-tests

Enforces that no class in the Domain or Application layer imports UnityEngine or SteamVR. Implemented via NetArchTest fitness functions. Any violation blocks the PR from merging.

Test name	Code example	Result	Reason
`Domain class has no Unity import`	`namespace iDaVIE.Domain // no using UnityEngine`	✓ **PASS**	Clean domain class
`Domain class imports UnityEngine`	`namespace iDaVIE.Domain using UnityEngine;`	✗ **FAIL**	Unity in domain layer: blocks merge
Infrastructure class imports Unity	`namespace iDaVIE.Infrastructure using UnityEngine;`	✓ **PASS**	Unity allowed in Infrastructure
Application class imports SteamVR	`namespace iDaVIE.Application using Valve.VR;`	✗ **FAIL**	SteamVR banned above Infrastructure

Gate 2: circular-deps

Runs Tarjan's Strongly Connected Components algorithm on the namespace dependency graph. Any cycle, regardless of how small, is a hard fail condition per Section 4.2 of the spec.

Test name	Dependency chain	Result	Reason
Downward dependency only	`Domain → Application → Infrastructure`	✓ **PASS**	Strict downward flow allowed
Infrastructure imports Domain	`Domain → Application → Infrastructure → Domain`	✗ **FAIL**	Cycle detected by Tarjan's
Two domain packages reference each other	`Domain.Feature ↔ Domain.Rendering`	✗ **FAIL**	Circular within same layer
Plugin calls back into Application	PluginHost → Application	✗ **FAIL**	Upward dependency, blocks merge

Gate 3: ck-metrics

Measures the Chidamber and Kemerer (CK) suite on every changed class. Thresholds are drawn directly from spec Section 7.1: WMC ≤ 20 (domain), CBO ≤ 14 (domain), RFC ≤ 50, LCOM ≤ 0.5, DIT ≤ 4.

Test name	Measured value	Result	Reason
VolumeMaterialBinder WMC	`WMC = 8 (target ≤ 20)`	✓ PASS	Well within threshold
VolumeDataSetRenderer WMC before refactor)	`WMC ≈ 35 (target ≤ 20)`	✗ FAIL	Exceeds domain class limit
`Domain class CBO`	`CBO = 12 (target ≤ 14)`	✓ PASS	Within coupling limit
God-class LCOM	`LCOM = 0.8 (target ≤ 0.5)`	✗ FAIL	Low cohesion: class doing too much
nterface size ISP check	`InputProvider = 5 members target ≤ 7)`	✓ PASS	Interface correctly segregated

Gate 4: build-dotnet

Fail-fast compilation gate. All other jobs wait for this to pass before running. Also enforces the singleton ban via SonarQube custom rules, any new static Instance field introduced after Day 3 fails the build.

Test name	Code example	Result	Reason
Constructor injected dependency	`public FitsService(IFitsReader r){}`	✓ PASS	Correct DI pattern
New singleton introduced	`public static Config Instance;`	✗ FAIL	SonarQube singleton rule, blocks merge
Code does not compile	Missing interface method implementation	✗ FAIL	Fail-fast gate — all other jobs skip

Gate 5: diagram-validation

Checks PlantUML and Mermaid syntax on any .puml or .mmd file changed in the PR. This gate is advisory only, it produces warnings but never blocks a merge. The human-review-required gate handles blocking for diagram changes.

Test name	File	Result	Reason
Valid PlantUML diagram	Layered-arch.puml: syntax OK	✓ PASS	Renders correctly
Malformed PlantUML	C4-model.puml: missing @enduml	⚠ WARN	Warning only, does not block merge

---

Summary — what blocks a PR vs what is advisory

Job	Blocks merge?	Notes
`build-dotnet`	YES	Fail-fast gate - everything else waits for this
`arch-tests`	YES	NetArchTest layer rules + UnityEngine/SteamVR ban
`ck-metrics`	YES	WMC≤20, CBO≤14, RFC≤50, LCOM≤0.5
`circular-deps`	YES	Tarjan's SCC on namespace graph
`sonar`	YES	Skipped if SONAR_TOKEN absent
`human-review-required`	YES	Fires on any .puml/.mmd/.drawio/.md/.pdf change
`diagram-validation`	NO	Warnings only, continue-on-error: true
`unity-tests`	NO	EditMode + PlayMode, continue-on-error so both run
`build-unity`	NO	Skipped if UNITY_LICENSE absent
`sprint-metrics`	NO	Push-to-main only, commits JSON history
